

THE APPLIED AI ENGINEER

A Production-Ready Guide

Image Analysis, Video Analytics & OCR with Azure AI Vision

What You Will Learn

- ✦ Azure AI Vision architecture and core capabilities
- ✦ Image analysis and object detection workflows
- ✦ Real-time video analytics with AI Video Indexer
- ✦ Optical Character Recognition (OCR) from images
- ✦ Full Python SDK implementation with code walkthroughs

Prerequisites: Python 3.8+ · Azure Subscription · Basic ML Understanding

01

Theory — Understanding Azure AI Vision

1. What is Azure AI Vision?

Azure AI Vision is a cloud-based cognitive service from Microsoft Azure that gives your applications the ability to analyse images and videos without you needing to build, train, or maintain your own computer vision models from scratch. Think of it as hiring a highly trained visual analyst who is available 24/7 via a simple API call — you send an image or video, and in return you receive structured, actionable insights.

In the context of GenAI and Data Science production systems, Azure AI Vision sits at the perception layer of your AI pipeline — it is the component that transforms raw visual data (images, frames, documents, video streams) into machine-readable information that downstream models or business logic can consume.

1.1 The Core Problem It Solves

Building a reliable computer vision model from scratch requires:

- Thousands (often millions) of labelled training images
- Significant compute resources for training (GPUs/TPUs over days or weeks)
- Deep expertise in model architecture (CNNs, ViTs, object detection heads)
- Ongoing maintenance as production data drifts over time

Azure AI Vision abstracts all of this complexity behind a REST API. Microsoft has already trained these models on massive, diverse datasets — your job is simply to call the right endpoint with the right parameters.

1.2 Architecture Overview

Azure AI Vision is part of the broader Azure AI Services umbrella (formerly Cognitive Services). Here is how the service hierarchy is structured:

Layer	Component	Your Touchpoint
Azure Cloud	Azure AI Services (multi-service account)	Azure Portal / ARM templates
Service Group	Vision + Document Intelligence	Azure AI Studio
Capability	Image Analysis OCR Face Video Indexer	REST API / Python SDK
Your App	Production ML Pipeline / Web App / Notebook	azure-ai-vision-imageanalysis SDK

1.3 Key Capabilities Deep Dive

1.3.1 Image Analysis

Image Analysis is the flagship capability. When you send an image to the endpoint, the service can simultaneously return multiple types of insights in a single API call:

Feature	What It Returns
Auto-Captioning	A human-readable sentence describing the entire scene (e.g., 'A person riding a bicycle in a park')
Dense Captioning	Multiple region-specific captions across different parts of the image
Object Detection	Bounding box coordinates + label + confidence score for each detected object
Image Tagging	A ranked list of keywords/tags describing image content (e.g., 'outdoor', 'bicycle', 'nature')
Smart Cropping	Suggests the most visually interesting crop coordinates for a given aspect ratio
Background Removal	Separates the foreground subject from the background (returns a masked PNG)
People Detection	Locates and counts people in images without facial recognition

1.3.2 Real-Time Video Analytics (Azure AI Video Indexer)

Azure AI Video Indexer extends vision capabilities into the temporal dimension — analysing video frame by frame and across time. It is particularly powerful for:

- Spatial analysis: Understanding physical spaces — tracking movement, counting people entering/leaving a zone, detecting crowd density
- Transcript generation: Extracting spoken audio to text with speaker diarisation (who said what)
- Face recognition: Identifying known faces across video frames (requires explicit opt-in)
- Object tracking: Following specific objects across frames
- Scene segmentation: Identifying topic changes and chapter boundaries in long-form video

From an architecture standpoint, Video Indexer deploys as a separate Azure resource that can optionally be linked to an Azure OpenAI resource — enabling generative summarisation of video content after the base analysis is complete.

1.3.3 Optical Character Recognition (OCR)

OCR is one of the most practically impactful capabilities in enterprise data science workflows. Azure AI Vision's Read API (the modern OCR engine) supports:

- Printed text in 164 languages
- Handwritten text in English, Chinese Simplified, French, German, Italian, Japanese, Korean, Portuguese, and Spanish
- Mixed-language documents in a single image
- Varied fonts, background colours, rotated text, and low-resolution images
- Hierarchical output: Page → Line → Word, each with bounding polygon coordinates

This is the capability that turns scanned invoices, identity documents, handwritten forms, and sign boards into structured, searchable, database-friendly text — directly powering document intelligence pipelines.

1.4 When to Use Azure AI Vision vs. Custom Models

A common architectural question is: should I use Azure AI Vision's pre-built models, or train my own?

Use Azure AI Vision When...	Consider Custom Models When...
Your task matches common visual categories (people, objects, text, scenes)	You need to detect highly domain-specific objects (e.g., rare industrial components)
You need rapid prototyping with minimal ML expertise on the team	Your data is sensitive and cannot leave your environment
Scale and reliability are critical and managed infra is preferred	You require sub-5ms latency and must run on-device
Multi-modal output (caption + objects + tags) from a single call is valuable	Your accuracy requirement exceeds what pre-trained APIs deliver

1.5 Pricing Model (Key Points for Production Planning)

Azure AI Vision uses a pay-per-transaction model. Understanding this is critical when designing production pipelines:

- Image Analysis: Priced per 1,000 transactions; Standard S1 tier unlocks higher quotas
- OCR (Read API): Priced per page for documents; free tier available for development
- Video Indexer: Priced per minute of video indexed; separate free and standard tiers
- All services offer a free tier for experimentation — use this during development to avoid unexpected charges

Production Tip

Always implement request batching, caching (store analysis results in a database so you don't re-analyse the same image twice), and circuit breakers in your production pipeline. A single unoptimised loop analysing the same 10,000 images daily can generate \$300–\$500/month unnecessarily.

1.6 Summary of Key Concepts

Before moving to the practical implementation, let's consolidate the mental model:

- Azure AI Vision = A set of pre-trained, hosted, production-grade computer vision models accessible via REST API
- Image Analysis = Multi-feature visual understanding of static images in a single call
- Video Indexer = Temporal video understanding including transcripts, spatial analytics, and scene detection
- OCR (Read API) = Extraction of printed and handwritten text into hierarchical, coordinate-aware structured output
- All capabilities are invocable from Python using the `azure-ai-vision-imageanalysis` and `azure-cognitiveservices-vision-computervision` SDKs

02

Practical — Setup, Demos & Code Walkthroughs

2. Environment Setup & Azure Resource Provisioning

Before writing a single line of Python, you need to provision the correct Azure resources. This section walks through both the Azure Portal method (for understanding) and the Azure CLI method (for production/IaC workflows).

2.1 Step 1 — Create an Azure AI Services Resource (Portal Method)

The Azure AI Services multi-service account gives you access to Vision, Language, Speech, and Translator all under a single API key and endpoint. This is the recommended starting point.

1. Log in to portal.azure.com and click 'Create a resource'
2. Search for 'Azure AI Services' and select the multi-service account option
3. Fill in the resource creation form with the following settings:

Field	Recommended Value
Subscription	Your active subscription (e.g., 'Main' or 'Pay-As-You-Go')
Resource Group	Create new → name it logically (e.g., 'rg-azure-ai-vision-dev')
Region	East US (lowest latency for most use cases; also broadest feature availability)
Name	A globally unique name (e.g., 'azure-ai-vision-yourname-dev')
Pricing Tier	Standard S0 for development; review limits before production

4. Click 'Review + Create', validate the summary, then click 'Create'
5. Wait for deployment to complete (typically 60–120 seconds)
6. Navigate to the resource → 'Keys and Endpoint' in the left sidebar
7. Copy Key 1 and the Endpoint URL — you will need these shortly

Security Note

Never hardcode API keys in your Python scripts or commit them to Git. Always use environment variables (`os.environ`), Azure Key Vault for production, or a `.env` file that is listed in your `.gitignore`.

2.2 Step 2 — Create an Azure AI Video Indexer Resource

Video Indexer is a separate resource type that must be provisioned independently:

8. In the Azure Portal search bar, type 'Azure AI Video Indexer' and select it
9. Click 'Create' and fill in the resource form:
 - Subscription and Resource Group: same as above for logical grouping
 - Resource Name: e.g., 'video-indexer-dev'
 - Region: East US (ensure it matches your other resources)
 - Storage Account: create a new storage account (required for video file storage)
10. Optionally link an Azure OpenAI resource under 'Advanced AI features' for generative summarisation
11. Accept the terms, click 'Review + Create', then 'Create'
12. Once deployed, navigate to the Video Indexer portal via the resource's 'Go to Azure Video Indexer' button

2.3 Step 3 — Python Environment Setup

Install the required packages in your virtual environment:

```
# Create and activate a virtual environment (recommended)
python -m venv .venv
source .venv/bin/activate          # Linux/macOS
.venv\Scripts\activate            # Windows

# Install Azure AI Vision SDK and supporting libraries
pip install azure-ai-vision-imageanalysis==1.0.0b3
pip install azure-cognitiveservices-vision-computervision
pip install msrest
pip install python-dotenv          # for .env file management
pip install Pillow                 # for local image handling
pip install requests               # for downloading sample images
```

Create a .env file in your project root to store credentials securely:

```
# .env (add this file to .gitignore immediately!)
AZURE_AI_VISION_ENDPOINT=https://your-resource-name.cognitiveservices.azure.com/
AZURE_AI_VISION_KEY=your_32_character_api_key_here
```

3. Demo 1 — Image Analysis with Azure AI Vision

This demo replicates the Azure AI Studio walkthrough from your notes, but goes further by implementing the same workflow programmatically in Python — giving you a reusable, production-ready pattern.

Cell 1 — Imports and Configuration

```
# Cell 1: Imports and Configuration
# -----
import os
from dotenv import load_dotenv
from azure.ai.vision.imageanalysis import ImageAnalysisClient
from azure.ai.vision.imageanalysis.models import VisualFeatures
from azure.core.credentials import AzureKeyCredential
from PIL import Image, ImageDraw, ImageFont
import requests
from io import BytesIO
import json

# Load environment variables from .env file
load_dotenv()

# Retrieve credentials — never hardcode these values
endpoint = os.environ['AZURE_AI_VISION_ENDPOINT']
key       = os.environ['AZURE_AI_VISION_KEY']

print(f'Endpoint configured: {endpoint}')
print('API Key loaded: ✓' if key else 'ERROR: API Key missing!')
```

Cell Explanation

We start by importing the `ImageAnalysisClient` — this is the main entry point for the Azure AI Vision SDK. The `VisualFeatures` enum contains constants for every type of analysis we can request (captions, objects, tags, etc.). `AzureKeyCredential` wraps our API key into the format the SDK expects. `load_dotenv()` reads our `.env` file and injects values into `os.environ` so they're accessible anywhere in the notebook without hardcoding.

Cell 2 — Initialise the Vision Client


```
# Cell 2: Initialise the Azure AI Vision Client
# -----
# Create a single client instance – reuse this across all API calls
# Creating a new client per call is expensive and an anti-pattern
client = ImageAnalysisClient(
    endpoint=endpoint,
    credential=AzureKeyCredential(key)
)

print('ImageAnalysisClient initialised successfully ✓')
```

Cell Explanation

The `ImageAnalysisClient` handles authentication, connection pooling, and retry logic for you. By instantiating it once and reusing it (rather than creating a new instance per request), you benefit from connection pooling — significantly reducing latency in loops where you process many images. In production, this client is typically initialised at app startup and injected as a dependency.

Cell 3 — Analyse an Image from a URL

```
# Cell 3: Analyse an Image from a URL
# -----
# Sample image URL – replace with your own
IMAGE_URL =
'https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg'

# Request multiple visual features in a single API call
# This is more efficient than making separate calls for each feature
result = client.analyze_from_url(
    image_url=IMAGE_URL,
    visual_features=[
        VisualFeatures.CAPTION,           # One-sentence image description
        VisualFeatures.OBJECTS,           # Detected objects + bounding boxes
        VisualFeatures.TAGS,              # Keyword tags
        VisualFeatures.DENSE_CAPTIONS,    # Region-specific captions
    ],
    gender_neutral_caption=True           # Use gender-neutral language in captions
)
```

```
print('Image analysis complete ✓')
```

Cell Explanation

The `analyze_from_url()` method accepts a publicly accessible image URL and a list of `VisualFeatures` to compute. Requesting multiple features in a single call is critical for efficiency — under the hood, the API runs all requested analyses in parallel on the server side, and you are billed as a single transaction rather than one transaction per feature. The `gender_neutral_caption=True` parameter instructs the captioning model to avoid gendered pronouns, which is important for fair and inclusive AI applications.

Cell 4 — Parse and Display Caption Results

```
# Cell 4: Parse and Display Caption Results
# -----
# The result object is a strongly-typed Python dataclass — no manual JSON parsing
# needed

# --- Main Caption ---
if result.caption is not None:
    print('=' * 60)
    print('MAIN CAPTION')
    print('=' * 60)
    print(f'  Description : {result.caption.text}')
    print(f'  Confidence   : {result.caption.confidence:.2%}')
    # Confidence of 0.85 means the model is 85% confident in this caption

# --- Dense Captions (region-specific descriptions) ---
if result.dense_captions is not None:
    print()
    print('=' * 60)
    print('DENSE CAPTIONS (Region-Specific)')
    print('=' * 60)
    for i, dc in enumerate(result.dense_captions.list):
        print(f'  Region {i+1}: {dc.text} (confidence: {dc.confidence:.2%})')
        print(f'          Bounding box: {dc.bounding_box}')
```

Cell Explanation

The result object uses Python dataclasses with auto-complete-friendly attribute access — `result.caption.text` is more readable and less error-prone than `result['caption']['text']`. The confidence score is a float between 0 and 1; formatting with `:.2%` converts it to a percentage string (e.g., 0.8734 becomes '87.34%'). Dense captions go a step further than the main caption by describing specific

rectangular regions of the image, each with its own `bounding_box` object containing `x`, `y`, `width`, and `height` in pixels.

Cell 5 — Parse Object Detection Results

```
# Cell 5: Parse and Display Object Detection Results
#
if result.objects is not None:
    print('=' * 60)
    print('DETECTED OBJECTS')
    print('=' * 60)
    for obj in result.objects.list:
        # Each object has a list of tags (labels) and a bounding box
        for tag in obj.tags:
            print(f'  Object   : {tag.name}')
            print(f'  Confidence: {tag.confidence:.2%}')
        bb = obj.bounding_box
        print(f'  Location: x={bb.x}, y={bb.y}, w={bb.width}, h={bb.height}')
        print()

# --- Tags (keyword-level labels for the whole image) ---
if result.tags is not None:
    print('=' * 60)
    print('IMAGE TAGS')
    print('=' * 60)
    # Tags are sorted by confidence descending by default
    for tag in result.tags.list:
        bar = '█' * int(tag.confidence * 20) # Visual confidence bar
        print(f'  {tag.name:<20} {bar} {tag.confidence:.2%}')
```

Cell Explanation

Object detection returns structured bounding box data — the pixel coordinates (`x`, `y`) of the top-left corner plus the width and height of the detected region. This is what you would feed into a drawing library (like PIL's `ImageDraw`) to overlay bounding boxes onto the original image for visualisation, or into a database to enable spatial querying. Tags differ from objects: tags describe the overall image semantically (like keywords), while objects are spatially located instances. Both are useful — tags for search indexing, objects for downstream spatial reasoning.

Cell 6 — Visualise Bounding Boxes on the Image

```

# Cell 6: Draw Bounding Boxes on the Original Image
# -----
# Download the original image
response = requests.get(IMAGE_URL)
img = Image.open(BytesIO(response.content)).convert('RGB')
draw = ImageDraw.Draw(img)

# Draw a bounding box for each detected object
if result.objects is not None:
    for obj in result.objects.list:
        bb = obj.bounding_box
        label = obj.tags[0].name if obj.tags else 'Unknown'
        confidence = obj.tags[0].confidence if obj.tags else 0

        # Draw rectangle: (x0, y0, x1, y1) format
        draw.rectangle(
            [bb.x, bb.y, bb.x + bb.width, bb.y + bb.height],
            outline='red',
            width=3
        )

        # Draw label text above the bounding box
        draw.text(
            (bb.x, bb.y - 20),
            f'{label} ({confidence:.0%})',
            fill='red'
        )

# Save and display the annotated image
img.save('annotated_output.jpg')
img.show() # Opens in default image viewer
print('Annotated image saved as annotated_output.jpg ✓')

```

Cell Explanation

This cell closes the loop between API output and visual insight — we take the raw bounding box coordinates from the API response and use PIL (Python Imaging Library) to draw coloured rectangles directly onto the original image. The `ImageDraw.rectangle()` method expects coordinates as (x0, y0, x1, y1), so we convert from the API's (x, y, width, height) format by computing $x1 = x + \text{width}$ and $y1 = y + \text{height}$.

$y + \text{height}$. This annotated visualisation pattern is essential when building datasets, validating model outputs, or presenting results to stakeholders.

4. Demo 2 — Real-Time Video Analytics with Azure AI Video Indexer

4.1 Azure AI Studio Method (UI Walkthrough)

For initial exploration, Azure AI Studio provides a no-code interface:

13. Log into portal.azure.com → search 'Azure AI Video Indexer' → open your resource
14. Click 'Go to Azure Video Indexer' to open the dedicated video portal
15. Click 'Upload' → select a video file from your local system (MP4, MOV, AVI supported)
16. Fill in video metadata: name, language, privacy setting (Private for development)
17. Click 'Upload + Index' and wait for indexing to complete (varies by video length — ~1× realtime for short videos)
18. Once complete, the portal displays: transcript, detected scenes, speaker diarisation, keywords, faces (if enabled), and a navigable timeline

4.2 Python SDK Method (Production Pattern)

```
# Cell 7: Video Indexer — Upload and Index a Video via Python
# -----
# Note: Video Indexer uses its own REST API (not the Vision SDK)
# Authenticate with your Video Indexer Account ID and API key

import requests
import time

VIDEO_INDEXER_ACCOUNT_ID = os.environ['VIDEO_INDEXER_ACCOUNT_ID']
VIDEO_INDEXER_API_KEY    = os.environ['VIDEO_INDEXER_API_KEY']
VIDEO_INDEXER_LOCATION   = 'trial' # or your Azure region, e.g., 'eastus'

BASE_URL = f'https://api.videoindexer.ai'

# Step 1: Get an access token (required for all subsequent calls)
def get_access_token():
    url =
    f'{BASE_URL}/auth/{VIDEO_INDEXER_LOCATION}/Accounts/{VIDEO_INDEXER_ACCOUNT_ID}/AccessToken'
    headers = {'Ocp-Apim-Subscription-Key': VIDEO_INDEXER_API_KEY}
    params  = {'allowEdit': 'true'}
```

```

response = requests.get(url, headers=headers, params=params)
response.raise_for_status()

# The token is returned as a quoted string – strip the quotes
return response.text.strip('\"')

access_token = get_access_token()
print('Access token obtained ✓')

```

Cell Explanation

Video Indexer uses its own authentication flow separate from the main Azure AI Vision SDK. The access token is a short-lived JWT (typically valid for 1 hour) that must accompany every subsequent API call in the Authorization header. The `get_access_token()` function should be called at the start of your session and the token refreshed before expiry in long-running pipelines. Note `raise_for_status()` — this is a best practice that immediately raises an exception if the HTTP response indicates an error (4xx or 5xx), preventing silent failures.

```

# Cell 8: Upload a Video for Indexing
# -----
def upload_video(video_url, video_name, access_token):
    """
    Upload a publicly accessible video URL for indexing.
    Returns the video_id assigned by Video Indexer.
    """
    url =
    f'{BASE_URL}/{VIDEO_INDEXER_LOCATION}/Accounts/{VIDEO_INDEXER_ACCOUNT_ID}/Videos'
    params = {
        'accessToken' : access_token,
        'name'        : video_name,
        'videoUrl'     : video_url,
        'privacy'      : 'Private',    # Never use Public for sensitive content
        'language'     : 'auto',       # Auto-detect spoken language
        'indexingPreset': 'Default',   # Balanced speed vs. depth
    }
    response = requests.post(url, params=params)
    response.raise_for_status()
    data = response.json()
    return data['id'] # This is your video_id for all future queries

# Use a publicly accessible video URL (or upload from local file via multipart)
SAMPLE_VIDEO_URL = 'https://www.learningcontainer.com/wp-content/uploads/2020/05/sample-mp4-file.mp4'

```

```
video_id = upload_video(SAMPLE_VIDEO_URL, 'sample-analysis', access_token)
print(f'Video uploaded. Video ID: {video_id}')
```

Cell Explanation

The `upload_video` function sends a POST request with query parameters (rather than a request body) — this is specific to the Video Indexer API convention. The `indexingPreset` parameter controls the depth of analysis: 'Default' gives transcript + faces + keywords; 'Advanced' adds audio effects, emotions, and observed people; 'BasicAudio' is fastest (transcript only). Choose based on your use case and cost constraints. The returned `video_id` is your handle for all subsequent queries — store it persistently.

```
# Cell 9: Poll for Indexing Completion and Extract Transcript
# -----
def wait_for_indexing(video_id, access_token, poll_interval=15):
    """
    Poll the Video Indexer API until processing is complete.
    Returns the full insight JSON when done.
    """
    url =
f'{BASE_URL}/{VIDEO_INDEXER_LOCATION}/Accounts/{VIDEO_INDEXER_ACCOUNT_ID}/Videos/{video_id}/Index'
    params = {'accessToken': access_token}

    while True:
        response = requests.get(url, params=params)
        data = response.json()
        state = data.get('state', 'unknown')
        print(f'  Indexing state: {state}')

        if state == 'Processed':
            return data # Full insight object
        elif state == 'Failed':
            raise RuntimeError(f'Video indexing failed: {data}')

        time.sleep(poll_interval) # Wait before next poll

insights = wait_for_indexing(video_id, access_token)

# Extract transcript lines
transcript = insights['videos'][0]['insights'].get('transcript', [])
print(f'\nTranscript extracted — {len(transcript)} segments found:')
for segment in transcript[:5]: # Print first 5 segments
```



```
print(f"  [{segment['instances'][0]['start']}] {segment['text']}")
```

Cell Explanation

Polling is the standard pattern for asynchronous operations like video indexing — you submit the job, then periodically check its status. The `poll_interval=15` seconds is a sensible default; too frequent polling wastes API quota, too infrequent delays your pipeline. In production, replace this polling loop with an Azure Event Grid webhook or a queue-based trigger so your service is notified upon completion rather than actively waiting. The transcript is returned as a list of segments, each with text content, speaker ID, and precise timestamp instances — ideal for building searchable video libraries or feeding into downstream LLM summarisation.

5. Demo 3 — OCR with Azure AI Vision (Read API)

OCR (Optical Character Recognition) transforms images containing text into machine-readable strings. This capability is the backbone of document intelligence pipelines — enabling you to ingest invoices, receipts, forms, and handwritten notes into your data systems.

Cell 10 — OCR with the Image Analysis Client

```
# Cell 10: Extract Text from an Image Using the Read API
# -----
# The same ImageAnalysisClient we initialised earlier handles OCR too
# Just add VisualFeatures.READ to your feature list

# Sample image with text (the bicycle sign image from your notes)
OCR_IMAGE_URL = 'https://learn.microsoft.com/azure/ai-services/computer-
vision/media/quickstarts/presentation.png'

ocr_result = client.analyze_from_url(
    image_url=OCR_IMAGE_URL,
    visual_features=[VisualFeatures.READ], # READ = OCR feature
)

print('OCR analysis complete ✓')
```

Cell Explanation

The `VisualFeatures.READ` constant activates the Azure AI Vision Read API — the latest generation OCR engine, significantly more accurate than the legacy OCR endpoint. We're reusing the same `ImageAnalysisClient` from earlier — this demonstrates the unified SDK pattern where a single client handles multiple vision tasks. For local image files (rather than URLs), use `client.analyze()` with an open file handle instead of `analyze_from_url()`.

```
# Cell 11: Parse and Display OCR Results (Hierarchical Structure)
# -----

if ocr_result.read is not None:
    print('=' * 60)
    print('EXTRACTED TEXT (Hierarchical View)')
    print('=' * 60)

    # The OCR result is structured as: Pages → Lines → Words
    for page in ocr_result.read.pages:
```

```

print(f'\nPage {page.page_number} | Angle: {page.angle:.1f}°')
print(f'Dimensions: {page.width:.0f}px × {page.height:.0f}px')
print('-' * 40)

for line in page.lines:
    # Full line text (convenient for most use cases)
    print(f'  LINE: {line.content}')

    # Word-level breakdown (useful for form field extraction)
    for word in line.words:
        # Each word has content, confidence, and bounding polygon
        polygon_pts = [(p.x, p.y) for p in word.bounding_polygon]
        print(f'      Word: "{word.content}"
              f' | Confidence: {word.confidence:.2%}'
              f' | Polygon: {polygon_pts}')

```

Cell Explanation

The OCR output is organised in a three-level hierarchy: Pages contain Lines, and Lines contain Words. This structure mirrors how humans read documents and enables powerful downstream logic. For most extraction tasks, the `line.content` attribute gives you the full text of each line ready to use. The word-level breakdown becomes critical when you need to extract specific fields from forms (e.g., 'find the word to the right of the label Total:'), or when building layout-aware document parsers that use spatial position to understand document structure. The `bounding_polygon` (rather than a simple bounding box) handles rotated or perspective-distorted text accurately.

Cell 12 — Extract All Text as a Clean String

```

# Cell 12: Flatten OCR Output to a Single Clean String
# -----
# For most NLP downstream tasks, you want a flat text string
# This function collapses the hierarchy into readable text

def extract_full_text(ocr_result):
    """
    Flatten the OCR result hierarchy into a single string.
    Preserves line breaks within pages; separates pages with double newline.
    """
    if ocr_result.read is None:
        return ''

```

```

    pages_text = []
    for page in ocr_result.read.pages:
        lines_text = [line.content for line in page.lines]
        pages_text.append('\n'.join(lines_text))

    return '\n\n'.join(pages_text)

full_text = extract_full_text(ocr_result)
print('Extracted Text:')
print('-' * 60)
print(full_text)
print('-' * 60)
print(f'\nTotal characters extracted: {len(full_text)}')
print(f'Total words extracted: {len(full_text.split())}')

```

Cell Explanation

This utility function is something you will reuse across nearly every OCR pipeline. It collapses the three-level hierarchy into a single string suitable for passing to downstream NLP models, storing in a database text column, or feeding into a retrieval-augmented generation (RAG) pipeline as a document chunk. The separation of pages with double newlines preserves multi-page structure while still producing a single string. In production, you would typically also want to store the raw structured result (with coordinates) in a separate column for cases where you need spatial re-analysis later.

Cell 13 — Process a Local Image File

```

# Cell 13: Analyse a Local Image File (not a URL)
# -----
# When working with proprietary or sensitive images that cannot be hosted
# publicly, use analyze() with a binary file stream instead of analyze_from_url()

LOCAL_IMAGE_PATH = 'sample_invoice.jpg' # Replace with your local file path

# Check if file exists before attempting analysis
if not os.path.exists(LOCAL_IMAGE_PATH):
    print(f'File not found: {LOCAL_IMAGE_PATH}')
    print('Download a sample: https://aka.ms/vision-read-sample')
else:
    # Open file in binary read mode and pass the stream to the SDK
    with open(LOCAL_IMAGE_PATH, 'rb') as image_file:

```

```
local_result = client.analyze(  
    image_data=image_file.read(),    # Reads entire file into memory  
    visual_features=[VisualFeatures.READ],  
)  
  
local_text = extract_full_text(local_result)  
print('Extracted text from local file:')  
print(local_text)
```

Cell Explanation

The `client.analyze()` method (note: no `_from_url` suffix) accepts raw binary image data via the `image_data` parameter. Reading the entire file into memory with `image_file.read()` is appropriate for typical document images (< 20MB). For very large images, you would stream the data in chunks or first resize/compress with PIL. This local file pattern is essential for production systems where images arrive from internal databases, file systems, or object storage (Azure Blob Storage) rather than public URLs.

6. Production Patterns & Best Practices

6.1 Batch Processing Pattern

Processing a single image at a time is fine for exploration, but production systems typically need to process thousands of images efficiently:

```
# Cell 14: Batch Image Processing with Error Handling and Retry
# -----
import asyncio
from azure.core.exceptions import HttpResponseError
import time

def analyse_with_retry(client, image_url, features, max_retries=3):
    """
    Analyse an image with exponential backoff retry on transient failures.
    Handles rate limiting (429) and transient server errors (5xx).
    """
    for attempt in range(max_retries):
        try:
            return client.analyze_from_url(
                image_url=image_url,
                visual_features=features
            )
        except HttpResponseError as e:
            if e.status_code == 429: # Rate limit hit
                wait_time = 2 ** attempt # Exponential: 1s, 2s, 4s
                print(f'Rate limited. Waiting {wait_time}s before retry {attempt+1}/{max_retries}')
                time.sleep(wait_time)
            elif attempt == max_retries - 1:
                raise # Re-raise on final attempt
            else:
                time.sleep(1)
    return None

# Process a batch of images
image_urls = [
    'https://example.com/image1.jpg',
```

```

    'https://example.com/image2.jpg',
    # ... add your image URLs
]

results = []
for i, url in enumerate(image_urls):
    print(f'Processing {i+1}/{len(image_urls)}: {url}')
    result = analyse_with_retry(
        client, url,
        [VisualFeatures.CAPTION, VisualFeatures.TAGS]
    )
    if result:
        results.append({'url': url, 'caption': result.caption.text,
                        'tags': [t.name for t in result.tags.list]})

print(f'\nProcessed {len(results)} images successfully')

```

Cell Explanation

The `analyse_with_retry` function implements the exponential backoff pattern — a critical production technique when dealing with rate-limited APIs. HTTP 429 (Too Many Requests) means you're sending requests faster than your subscription quota allows; exponential backoff (waiting 1s, then 2s, then 4s between retries) prevents a thundering herd that would keep hitting the limit. In high-throughput production systems, you would also implement a token bucket or sliding window rate limiter on your side to avoid hitting the API limit in the first place.

6.2 Summary Reference Card

Capability	SDK Method	Key Output Attribute
Image Caption	<code>client.analyze_from_url(..., [VisualFeatures.CAPTION])</code>	<code>result.caption.text</code>
Object Detection	<code>client.analyze_from_url(..., [VisualFeatures.OBJECTS])</code>	<code>result.objects.list[].bounding_box</code>
Image Tags	<code>client.analyze_from_url(..., [VisualFeatures.TAGS])</code>	<code>result.tags.list[].name</code>
OCR / Read	<code>client.analyze_from_url(..., [VisualFeatures.READ])</code>	<code>result.read.pages[].lines[].content</code>
Local File	<code>client.analyze(image_data=bytes, visual_features=[...])</code>	Same as URL methods
Video Indexer	REST API: POST /Videos, GET /Videos/{id}/Index	<code>insights.transcript[].text</code>

Next Chapter Preview

In the next chapter, we will build on these vision capabilities to construct a complete end-to-end document intelligence pipeline — combining OCR text extraction, Azure AI Language for entity recognition, and Azure OpenAI for generative summarisation. This is the foundation of modern intelligent document processing (IDP) systems deployed in finance, healthcare, and legal industries.